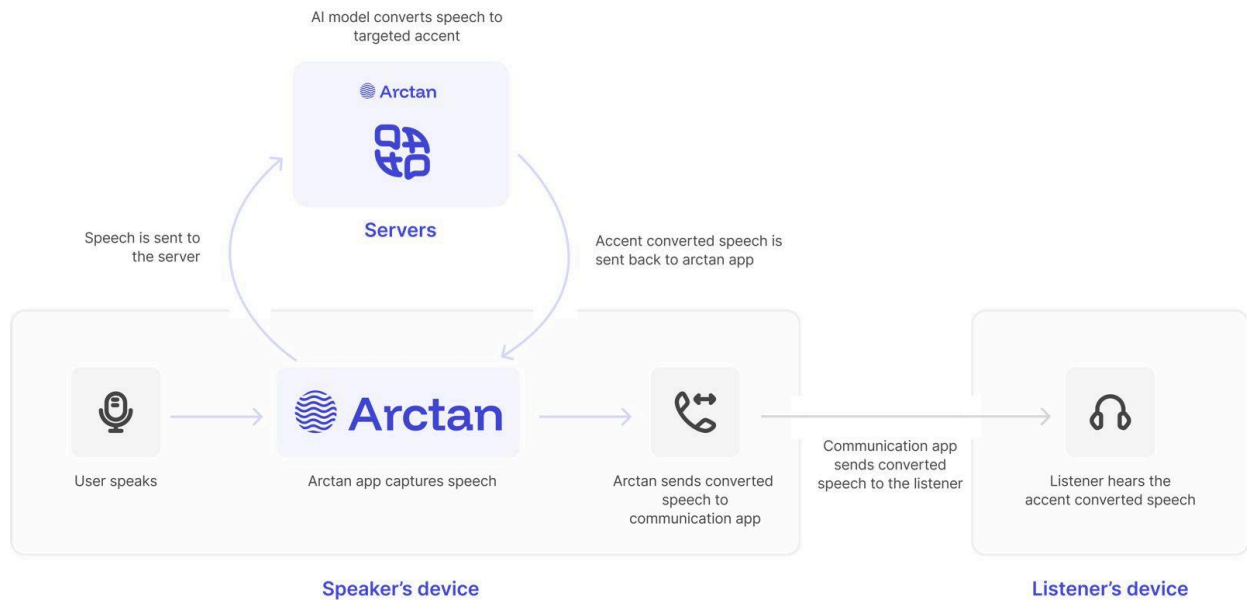


Technical Documentation

Overview:

Arctan is a Voice AI product designed for real-time speech-to-speech accent conversion. It aims to enhance global communication by converting speech from one accent to another in real-time to enhance understandability of the speaker.

Product:



- Our .NET-based desktop client captures microphone input with minimal latency.
- Audio is chunked & streamed to our GPU-backed inference server over persistent WebSocket connections.
- On the server, we run a real-time speech-to-speech pipeline via transformer based models which adjust the speaker's accent.
- The converted speech is streamed back to the desktop application and sent to Zoom, Google Meet, or Dialer environments using virtual audio driver enabling real-time audio replacement.

Architecture:

1. Model Architecture:

The pipeline is composed of modular components including a **speech encoder (HuBERT)**, **pitch extractor (RMVPE)**, **speech vector retriever (FAISS)**, and **neural vocoder (HiFi-GAN)**. HuBERT extract linguistic content and pitch information, retrieve the appropriate speaker embedding, and then synthesize the target-accented waveform.

- **HuBERT (Content Encoder):** HuBERT (Hidden Unit Bidirectional Encoder Representations from Transformers) extracts speech embeddings from the raw speech into mel-spectrogram like features. We use intermediate layer embeddings to preserve low-level acoustic features and high-level semantic representations generating a 756 dimensional speech embedding.
- **RMVPE (Pitch Extractor):** RMVPE (Robust Model for Vocal Pitch Estimation) extracts the fundamental frequency (f_0 contour) of the input speech. The extracted pitch is used as a conditioning input to the vocoder (HiFi-GAN), allowing the synthesized audio to retain the original speech's intonation, rhythm, and expressiveness.
- **FAISS Index (Accent Embedding Retrieval):**
 - FAISS (Facebook AI Similarity Search) is a high-speed vector similarity engine used to retrieve accent-style embeddings from a custom prebuilt index.
 - We've built custom FAISS indexes using curated speech data that emphasizes phonetic variations across speakers for a target accent enabling precise control over pronunciation transformations.
 - We use features extracted by HuBERT to query our custom FAISS indexes and retrieve the closest matching target accent embeddings, enabling the conversion of the input speaker's accent to the target accent while preserving the speech content.
 - By designing domain-specific accent indexes, we can optimize for specific use cases.
- **HiFi-GAN (Vocoder):** In our system, HiFi-GAN (High Fidelity Generative Adversarial Network) serves as the vocoder that reconstructs the final waveform by conditioning on accent-modified features.
 - We input speech embeddings derived via HuBERT blended with accent-style vectors retrieved from our custom FAISS index. To preserve pitch and emotional nuance, we incorporate pitch contour (f_0) extracted using RMVPE, ensuring that intonation and expressiveness remain natural.

- Our adaptation of HiFi-GAN uses its generator architecture particularly its multi-receptive field fusion (MRF) and multi-period discriminators (MPD) not just for quality, but to maintain accent accuracy and emotional fidelity generating human like voices.

2. Training Methodology:

- **Voice Synthesis Training (HiFi-GAN Fine-Tuning)**

We fine-tune HiFi-GAN on curated audio samples of a single target speaker.

- **Step 1:** Mel-spectrograms are extracted using a pre-trained HuBERT model to capture the linguistic and phonetic content of the speech.
- **Step 2:** These features are used to train the HiFi-GAN generator to reconstruct time-domain waveforms that replicate the timbre, prosody, and general characteristics of the target speaker, ensuring consistent voice identity across conversions.
- **Pitch Conditioning:** We also integrate RMVPE-derived f_0 contours into the training process to help the vocoder model pitch dynamics and emotional expressiveness accurately.

- **Accent Index Training (FAISS Index Construction)**

To capture the **accentual variations** across speakers, we construct custom FAISS indexes trained on voice data from English speakers representing the **target accent (e.g., American, British)**.

- **Feature Extraction:** From each audio sample, we extract both semantic and low-level acoustic features that encode pronunciation styles and prosodic cues (Intermediate layer embeddings).
- **Index Type:** We employ an **IVF (Inverted File)** index with **k-means clustering** to enable fast, scalable vector retrieval.
- **Index Construction:**
 - Cluster the embedding space into **Voronoi cells** (typically 100–1000 clusters).
 - Compute **centroids** for each cell to serve as anchors.
 - Build **inverted lists** that associate each feature vector with its nearest centroid.

During inference, input embeddings are matched against this index to retrieve the closest accent-style features, enabling accurate accent transformation without retraining the model.

3. Streaming Architecture:

- Desktop Application (Client): Our desktop application (built in .NET) acts as the real-time capture and playback interface. It performs:
 - Captures raw 16-bit PCM audio at 48kHz from the user's mic with low-latency buffers
 - Basic denoising and normalization applied locally for input consistency
 - Sends buffered audio chunks via a persistent WebSocket connection to our inference server
 - Receives converted audio from the server and plays it out using virtual audio routing.
- Server-Side Architecture: Hosted on GPU-powered EC2 instances with autoscaling capabilities, the server handles the core real-time inference:
 - The server accepts real-time audio streams over persistent WebSocket connections
 - A session-level controller coordinates the flow of audio through preprocessing, inference, and post processing modules with strict latency constraints.
 - Once generated, audio is chunked, and streamed back over the same WebSocket connection. Output buffers ensure continuity even in variable network conditions.
 - Hosted on g4dn.xlarge instances, with containerized deployment via Docker for portability and reproducibility.
 - Integrated logging, heartbeat pings, and timeouts allow us to recover from dropped or unstable client connections in real time.